

VSD HW2　5-stage Pipeline CPU Design

Instructor: Lih-Yih Chiou

TA: 涂宇宸 Eric

Date: 2026/10/5



Outline

- Introduction
- RISC-V "F" Extension
- 5-Stage Pipeline CPU
- Homework Explanation
- Test Programs
- Grading Policy
- Submission Guidelines

No part of this confidential report may be reproduced in any form without written permission from Prof. Lih-Yih Chiou NCKU LPHP Lab, Taiwan



LPHPLAB
VLSI Design LAB

Introduction

No part of this confidential report may be reproduced in any form without written permission from Prof.
Lih-Yih Chiou NCKU LPHP Lab, Taiwan



Introduction

- Design and implement a 5-stage pipelined RISC-V processor supporting the RV32IMBF+DSP instruction set.
- Understand how different CPU submodules cooperate through the datapath.
- Verify correct instruction execution through simulation.

No part of this confidential report may be reproduced in any form without written permission from Prof. Lih-Yih Chiou NCKU LPHP Lab, Taiwan



LPHPLAB
VLSI Design LAB

RISC-V "F" Extension

No part of this confidential report may be reproduced in any form without written permission from Prof.
Lih-Yih Chiou NCKU LPHP Lab, Taiwan

F Register State

- The F extension adds 32 floating-point registers, f0-f31, each 32 bits wide. **f0 is NOT hard-wired to zero.**
- In this project, you do **not** need to implement the floating-point control and status register (**fcsr**).
- Floating-point operations will not produce NaN, infinity, subnormal numbers, or overflow/underflow.
- Results may require rounding: **round-to-nearest, ties-to-even**.
- If the result of an add/sub is exactly zero (e.g., x − x), output +0 (32'h00000000).

register	name	register	name
f0	ft0	f16	fa6
f1	ft1	f17	fa7
f2	ft2	f18	fs2
f3	ft3	f19	fs3
f4	ft4	f20	fs4
f5	ft5	f21	fs5
f6	ft6	f22	fs6
f7	ft7	f23	fs7
f8	fs0	f24	fs8
f9	fs1	f25	fs9
f10	fa0	f26	fs10
f11	fa1	f27	fs11
f12	fa2	f28	ft8
f13	fa3	f29	ft9
f14	fa4	f30	ft10
f15	fa5	f31	ft11

No part of this confidential report may be reproduced in any form without written permissio
Lih-Yih Chiou NCKU LPHP Lab, Taiwan

F Extension Instruction

31 20	19 15	14 12	11 7	6 0		
imm[11:0]	**rs1**	**funct3**	**rd**	**opcode**	**Mnemonic**	**Description**
imm[11:0]	rs1	010	frd	0000111	FLW	frd = M[rs1+imm]

31 25	24 20	19 15	14 12	11 7	6 0		
imm[11:5]	**rs2**	**rs1**	**funct3**	**imm[4:0]**	**opcode**	**Mnemonic**	**Description**
imm[11:5]	frs2	rs1	010	imm[4:0]	0100111	FSW	M[rs1+imm] = frs2

31 27	26 25	24 20	19 15	14 12	11 7	6 0		
funct5	**fmt**	**rs2**	**rs1**	**rm funct3**	**rd**	**opcode**	**Mnemonic**	**Description**
00000	00	frs2	frs1	111	frd	1010011	FADD.S	frd = frs1 + frs2
00001	00	frs2	frs1	111	frd	1010011	FSUB.S	frd = frs1 - frs2
00101	00	frs2	frs1	000	frd	1010011	FMIN.S	frd=min(frs1, frs2)
00101	00	frs2	frs1	001	frd	1010011	FMAX.S	frd=max(frs1, frs2)

No part of this confidential report may be reproduced in any form without written permission from Prof. Lih-Yih Chiou NCKU LPHP Lab, Taiwan



LPHPLAB
VLSI Design LAB

5-Stage Pipeline CPU

No part of this confidential report may be reproduced in any form without written permission from Prof. Lih-Yih Chiou NCKU LPHP Lab, Taiwan



Introduction to CPU Modules

- **Decoder**: Translates instructions into control signals for CPU operations.
- **Imm_ext**: Extends immediate values to match the CPU's data width
- **LD_filter**: Filters and aligns data from memory for load instructions.
- **Reg_PC**: Holds the address of the current or next instruction.
- **Regfile**: Stores integer operands and results in 32 registers.
- **Regfile_F**: Stores floating-point operands and results in 32 registers.
- **ALU**: Performs arithmetic and logical operations on operands.
- **JB_unit**: Manages jump and branch instructions for control flow.
- **Controller**: Coordinates and sequences operations of all CPU modules.
- **CSR**: Manages processor-level control and status registers.

No part of this confidential report may be reproduced in any form without written permission from Prof.
Lih-Yih Chiou NCKU LPHP Lab, Taiwan



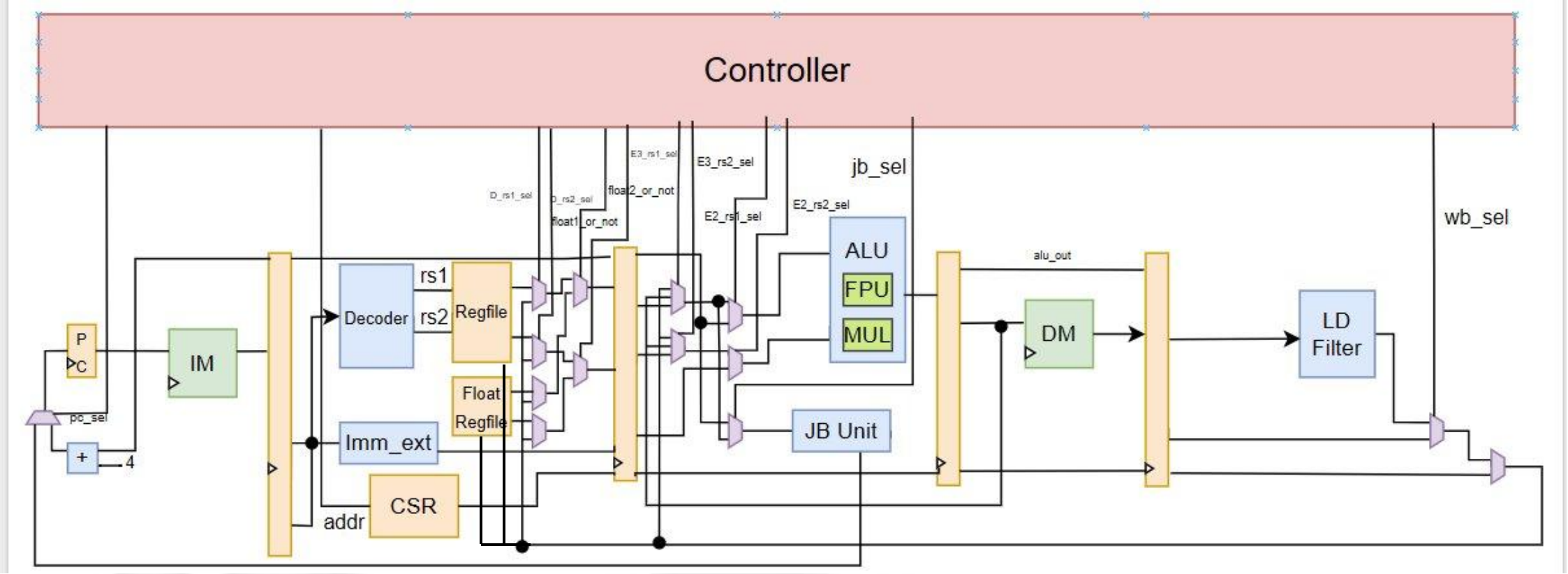
5 Stage Pipeline CPU

Our CPU adopts a classic 5-stage pipeline to improve instruction throughput:

- IF (Instruction Fetch)
  - ➔ Fetch the next instruction from instruction memory based on the current PC.
- ID (Instruction Decode & Register Read)
  - ➔ Decode the instruction type and read source operands from the register file.
- EX (Execute)
  - ➔ Perform ALU operations, address calculation, branch comparison, or MAC/FPU operations.
- MEM (Memory Access)
  - ➔ Access data memory for load/store instructions.
- WB (Write Back)
  - ➔ Write the computed result back to the destination register.

No part of this confidential report may be reproduced in any form without written permission from Prof. Lih-Yih Chiou NCKU LPHP Lab, Taiwan

5 Stage Pipeline CPU

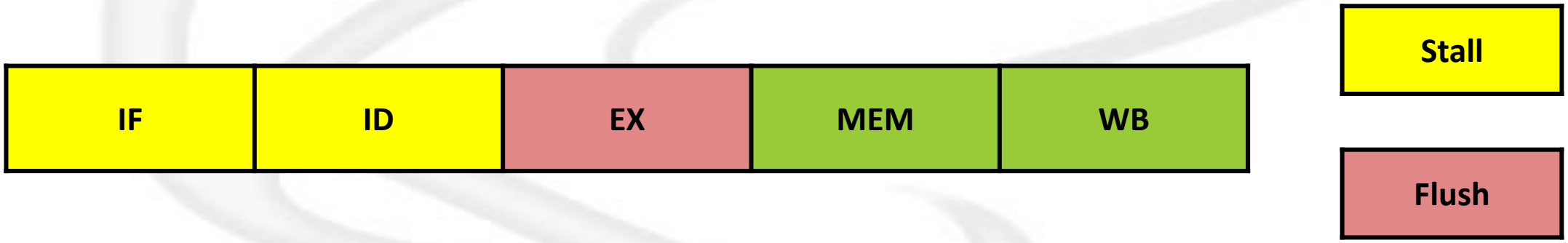


The architecture diagram is for reference only.

No part of this confidential report may be reproduced in any form without written permission from Prof.
Lih-Yih Chiou NCKU LPHP Lab, Taiwan

Stall & Flush in Pipeline

- Stall
  - When the next instruction cannot safely proceed (e.g., waiting for load result or hazard resolution), the pipeline inserts bubbles and temporarily stops some stages. This keeps correctness at the cost of performance.
- Flush
  - When a mispredicted branch or exception occurs, the wrong-path instructions already in the pipeline are discarded (flushed).



No part of this confidential report may be reproduced in any form without written permission from Prof. Lih-Yih Chiou NCKU LPHP Lab, Taiwan



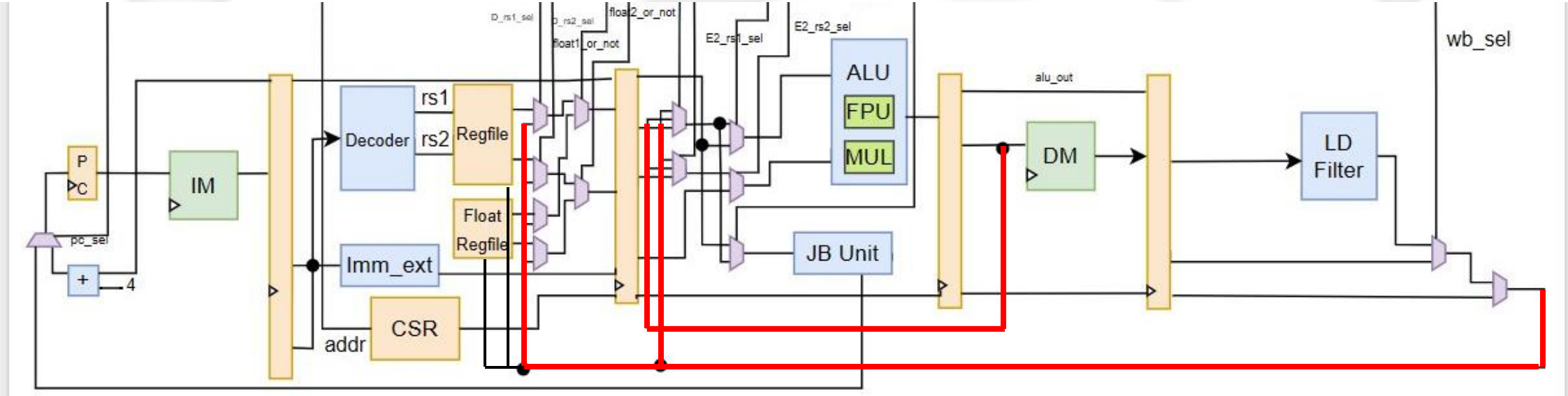
Data Hazards

- A **data hazard** occurs when an instruction needs a register value that is **still being produced** by a previous instruction in the pipeline.
- In a 5-stage pipeline (IF, ID, EX, MEM, WB), this is usually a **RAW (Read After Write)** dependency.
- Example 1:
  - add x3, x1, x2
  - sub x5, x3, x4 ← sub needs x3, which is still being computed
- Example 2:
  - lw x3, 0(x1)
  - add x5, x3, x4 ← add needs x3, which is still loading

No part of this confidential report may be reproduced in any form without written permission from Prof. Lih-Yih Chiou NCKU LPHP Lab, Taiwan

Type 1: Hazards Solved by Forwarding

- For "**ALU → ALU**" dependencies (add, sub, logic ops), the result is already available at the end of the **EX stage**.
- We use **forwarding / bypassing** to send the ALU result directly to the next instruction's EX inputs, **without waiting** for WB.
- This removes the need for a stall: the pipeline can keep issuing one instruction per cycle.



No part of this confidential report may be reproduced in any form without written permission from Prof. Lih-Yih Chiou NCKU LPHP Lab, Taiwan

Type 2: Load-Use Hazard

- For a load instruction, the data is only available after the **MEM stage**.
- If the **very next instruction** uses the loaded register, we get a **load-use hazard**
- Even with forwarding, the value from MEM is not ready in time for the next EX stage.
- Solution: **insert one bubble / stall one cycle** so that the add executes only after the load data is available.

No part of this confidential report may be reproduced in any form without written permission from Prof. Lih-Yih Chiou NCKU LPHP Lab, Taiwan

Handling Long-Latency IM/DM in a 5-Stage CPU

- Instruction Fetch (IF) with long IM latency
  - **Freezes PC** (no update).
  - **Freezes IF/ID register** (no new instruction).
  - The whole pipeline gradually stalls while waiting for the next instruction.
- MEM stage with long DM latency
  - While data not ready or write not accepted, the **instruction stays in MEM,** and earlier pipeline stages are **stalled**.

No part of this confidential report may be reproduced in any form without written permission from Prof. Lih-Yih Chiou NCKU LPHP Lab, Taiwan



Control Hazards

- The Pipeline Branch Problem
  - **Flow of Control:** Determined by the branch outcome.
  - **Fetch Dependency:** Fetching the next instruction relies on the branch result, which is unknown early, so the pipeline **can't always fetch the correct instruction**.
  - **Resolution Delay:** The branch decision (comparing registers) often happens late, even while the instruction is still working on the **ID (Instruction Decode) stage**.
- RISC-V Optimization Strategy
  - **Early Requirement:** The pipeline **needs to compare registers and compute the target address** very early.
  - **ID Stage Modification: Add hardware** to the **ID stage** to perform both the register comparison and target address calculation.

No part of this confidential report may be reproduced in any form without written permission from Prof. Lih-Yih Chiou NCKU LPHP Lab, Taiwan



LPHPLAB
VLSI Design LAB

Homework Explanation

No part of this confidential report may be reproduced in any form without written permission from Prof.
Lih-Yih Chiou NCKU LPHP Lab, Taiwan



CPU Requirement

- Implements the RISC-V ISA with the specified 68 instructions (including Base Integer, M, B, F, and DSP extensions).
- Pipeline depth is at least 5 stages.
- Integer register file: fixed 32 × 32-bit general-purpose registers (x0 hard-wired to 0).
- Floating-point register file: fixed 32 × 32-bit registers.
- Program counter: 32-bit.
- Max clock period 2 ns
- The hardware design must not violate any of the above specifications.

No part of this confidential report may be reproduced in any form without written permission from Prof. Lih-Yih Chiou NCKU LPHP Lab, Taiwan

Instructions R-Type

31 25	24 20	19 15	14 12	11 7	6 0		
funct7	**rs2**	**rs1**	**funct3**	**rd**	**opcode**	**Mnemonic**	**Description**
0000000	rs2	rs1	000	rd	0110011	ADD	rd = rs1 + rs2
0100000	rs2	rs1	000	rd	0110011	SUB	rd = rs1 - rs2
0000000	rs2	rs1	001	rd	0110011	SLL	rd = $rs1_u$ << rs2[4:0]
0000000	rs2	rs1	010	rd	0110011	SLT	rd = ($rs1_s$ < $rs2_s$)? 1:0
0000000	rs2	rs1	011	rd	0110011	SLTU	rd = ($rs1_u$ < $rs2_u$)? 1:0
0000000	rs2	rs1	100	rd	0110011	XOR	rd = rs1 ^ rs2
0000000	rs2	rs1	101	rd	0110011	SRL	rd = $rs1_u$ >> rs2[4:0]
0100000	rs2	rs1	101	rd	0110011	SRA	rd = $rs1_s$ >> rs2[4:0]
0000000	rs2	rs1	110	rd	0110011	OR	rd = rs1 | rs2
0000000	rs2	rs1	111	rd	0110011	AND	rd = rs1 & rs2

No part of this confidential report may be reproduced in any form without written permission from Prof. Lih-Yih Chiou NCKU LPHP Lab, Taiwan

Instructions M-extension, J-Type

31 25	24 20	19 15	14 12	11 7	6 0		
funct7	**rs2**	**rs1**	**funct3**	**rd**	**opcode**	**Mnemonic**	**Description**
0000001	rs2	rs1	000	rd	0110011	MUL	rd = lower 32 bits of (rs1 * rs2)
0000001	rs2	rs1	001	rd	0110011	MULH	rd = upper 32 bits of (rs1 * rs2) signed * signed
0000001	rs2	rs1	010	rd	0110011	MULHSU	rd = upper 32 bits of (rs1 * rs2) signed * unsigned
0000001	rs2	rs1	011	rd	0110011	MULHU	rd = upper 32 bits of (rs1 * rs2) unsigned * unsigned

31 12	11 7	6 0		
imm[20|10:1|11|19:12]	**rd**	**opcode**	**Mnemonic**	**Description**
imm[20|10:1|11|19:12]	rd	1101111	JAL	rd = PC + 4 PC = PC + imm

No part of this confidential report may be reproduced in any form without written permission from Prof. Lih-Yih Chiou NCKU LPHP Lab, Taiwan

Instructions I-Type

31	20	19	15	14	12	11	7	6	0		
imm[11:0]		**rs1**		**funct3**		**rd**		**opcode**		**Mnemonic**	**Description**
imm[11:0]		rs1		010		rd		0000011		LW	rd = M[rs1+imm]
imm[11:0]		rs1		000		rd		0010011		ADDI	rd = rs1 + imm
imm[11:0]		rs1		010		rd		0010011		SLTI	rd = ($rs1_s$ < imm_s)? 1:0
imm[11:0]		rs1		011		rd		0010011		SLTIU	rd = ($rs1_u$ < imm_u)? 1:0
imm[11:0]		rs1		100		rd		0010011		XORI	rd = rs1 ^ imm
imm[11:0]		rs1		110		rd		0010011		ORI	rd = rs1 | imm
imm[11:0]		rs1		111		rd		0010011		ANDI	rd = rs1 & imm
imm[11:0]		rs1		000		rd		0000011		LB	rd = M[rs1+imm]bs
0000000	shamt	rs1		001		rd		0010011		SLLI	rd = $rs1_u$ << shamt
0000000	shamt	rs1		101		rd		0010011		SRLI	rd = $rs1_u$ >> shamt
0100000	shamt	rs1		101		rd		0010011		SRAI	rd = $rs1_s$ >> shamt
imm[11:0]		rs1		000		rd		1100111		JALR	rd = PC + 4 PC = imm + rs1 (Set LSB of PC to 0)
imm[11:0]		rs1		001		rd		0000011		LH	rd = $M[rs1+imm]_{hs}$
imm[11:0]		rs1		100		rd		0000011		LBU	rd = $M[rs1+imm]_{bu}$
imm[11:0]		rs1		101		rd		0000011		LHU	rd = $M[rs1+imm]_{hu}$

No part of this confidential report may be reproduced in any form without written permission from Prof. Lih-Yih Chiou NCKU LPHP Lab, Taiwan

Instructions S-Type, U-Type

31 25	24 20	19 15	14 12	11 7	6 0		
imm[11:5]	**rs2**	**rs1**	**funct3**	**imm[4:0]**	**opcode**	**Mnemonic**	**Description**
imm[11:5]	rs2	rs1	010	imm[4:0]	0100011	SW	M[rs1+imm] = rs2
imm[11:5]	rs2	rs1	000	imm[4:0]	0100011	SB	M[rs1+imm]b = $rs2_b$
imm[11:5]	rs2	rs1	001	imm[4:0]	0100011	SH	$M[rs1+imm]_h = rs2_h$

31 12	11 7	6 0		
imm[31:12]	**rd**	**opcode**	**Mnemonic**	**Description**
imm[31:12]	rd	0010111	AUIPC	rd = PC + imm
imm[31:12]	rd	0110111	LUI	rd = imm

No part of this confidential report may be reproduced in any form without written permission from Prof. Lih-Yih Chiou NCKU LPHP Lab, Taiwan

Instructions B-Type

31 25	24 20	19 15	14 12	11 7	6 0		
imm[12|10:5]	**rs2**	**rs1**	**funct3**	**imm[4:1|11]**	**opcode**	**Mnemonic**	**Description**
imm[12|10:5]	rs2	rs1	000	imm[4:1|11]	1100011	BEQ	PC = (rs1 == rs2)? PC + imm: PC + 4
imm[12|10:5]	rs2	rs1	001	imm[4:1|11]	1100011	BNE	PC = (rs1 != rs2)? PC + imm: PC + 4
imm[12|10:5]	rs2	rs1	100	imm[4:1|11]	1100011	BLT	PC = ($rs1_s$ < $rs2_s$)? PC + imm: PC + 4
imm[12|10:5]	rs2	rs1	101	imm[4:1|11]	1100011	BGE	PC = ($rs1_s$ ≥ $rs2_s$)? PC + imm: PC + 4
imm[12|10:5]	rs2	rs1	110	imm[4:1|11]	1100011	BLTU	PC = ($rs1_u$ < $rs2_u$)? PC + imm: PC + 4
imm[12|10:5]	rs2	rs1	111	imm[4:1|11]	1100011	BGEU	PC = ($rs1_u$ ≥ $rs2_u$)? PC + imm: PC + 4

No part of this confidential report may be reproduced in any form without written permission from Prof. Lih-Yih Chiou NCKU LPHP Lab, Taiwan

Instructions CSR

31 20	19 15	14 12	11 7	6 0		
imm[11:0]	**rs1**	**funct3**	**rd**	opcode	**Mnemonic**	**Description**
110010000010	00000	010	rd	1110011	RDINSTRETH	rd = instret[63:32]
110000000010	00000	010	rd	1110011	RDINSTRET	rd = instret[31:0]
110010000000	00000	010	rd	1110011	RDCYCLEH	rd = cycle[63:32]
110000000000	00000	010	rd	1110011	RDCYCLE	rd = cycle[31:0]

No part of this confidential report may be reproduced in any form without written permission from Prof. Lih-Yih Chiou NCKU LPHP Lab, Taiwan

Instructions F-extension

31 20	19 15	14 12	11 7	6 0		
imm[11:0]	**rs1**	**funct3**	**rd**	**opcode**	**Mnemonic**	**Description**
imm[11:0]	rs1	010	frd	0000111	FLW	frd = M[rs1+imm]

31 25	24 20	19 15	14 12	11 7	6 0		
imm[11:5]	**rs2**	**rs1**	**funct3**	**imm[4:0]**	**opcode**	**Mnemonic**	**Description**
imm[11:5]	frs2	rs1	010	imm[4:0]	0100111	FSW	M[rs1+imm] = frs2

31 27	26 25	24 20	19 15	14 12	11 7	6 0		
funct5	**fmt**	**rs2**	**rs1**	**rm funct3**	**rd**	**opcode**	**Mnemonic**	**Description**
00000	00	frs2	frs1	111	frd	1010011	FADD.S	frd = frs1 + frs2
00001	00	frs2	frs1	111	frd	1010011	FSUB.S	frd = frs1 - frs2
00101	00	frs2	frs1	000	frd	1010011	FMIN.S	frd=min(frs1, frs2)
00101	00	frs2	frs1	001	frd	1010011	FMAX.S	frd=max(frs1, frs2)

No part of this confidential report may be reproduced in any form without written permission from Prof. Lih-Yih Chiou NCKU LPHP Lab, Taiwan

Instructions B-extension

31 25	24 20	19 15	14 12	11 7	6 0		
funct7	**rs2**	**rs1**	**funct3**	**rd**	**opcode**	**Mnemonic**	**Description**
0110000	00000	rs1	001	rd	0010011	CLZ	rd = count_leading_zeros(rs1)
0110000	00001	rs1	001	rd	0010011	CTZ	rd = count_trailing_zeros(rs1)
0110000	00010	rs1	001	rd	0010011	CPOP	rd = count_ones(rs1)
0110000	rs2	rs1	001	rd	0110011	ROL	rd = rotate_left(rs1, rs2[4:0])
0110000	rs2	rs1	101	rd	0110011	ROR	rd = rotate_right(rs1, rs2[4:0])
0000101	rs2	rs1	100	rd	0110011	MIN	rd = (signed(rs1) < signed(rs2)) ? rs1 : rs2
0000101	rs2	rs1	110	rd	0110011	MAX	rd = (signed(rs1) > signed(rs2)) ? rs1 : rs2
0000101	rs2	rs1	101	rd	0110011	MINU	rd = (unsigned(rs1) < unsigned(rs2)) ? rs1 : rs2
0000101	rs2	rs1	111	rd	0110011	MAXU	rd = (unsigned(rs1) > unsigned(rs2)) ? rs1 : rs2
0010100	rs2	rs1	001	rd	0110011	BSET	rd = rs1 | (1 << rs2[4:0])
0100100	rs2	rs1	001	rd	0110011	BCLR	rd = rs1 & ~(1 << rs2[4:0])
0100100	rs2	rs1	101	rd	0110011	BEXT	rd = (rs1 >> rs2[4:0]) & 1

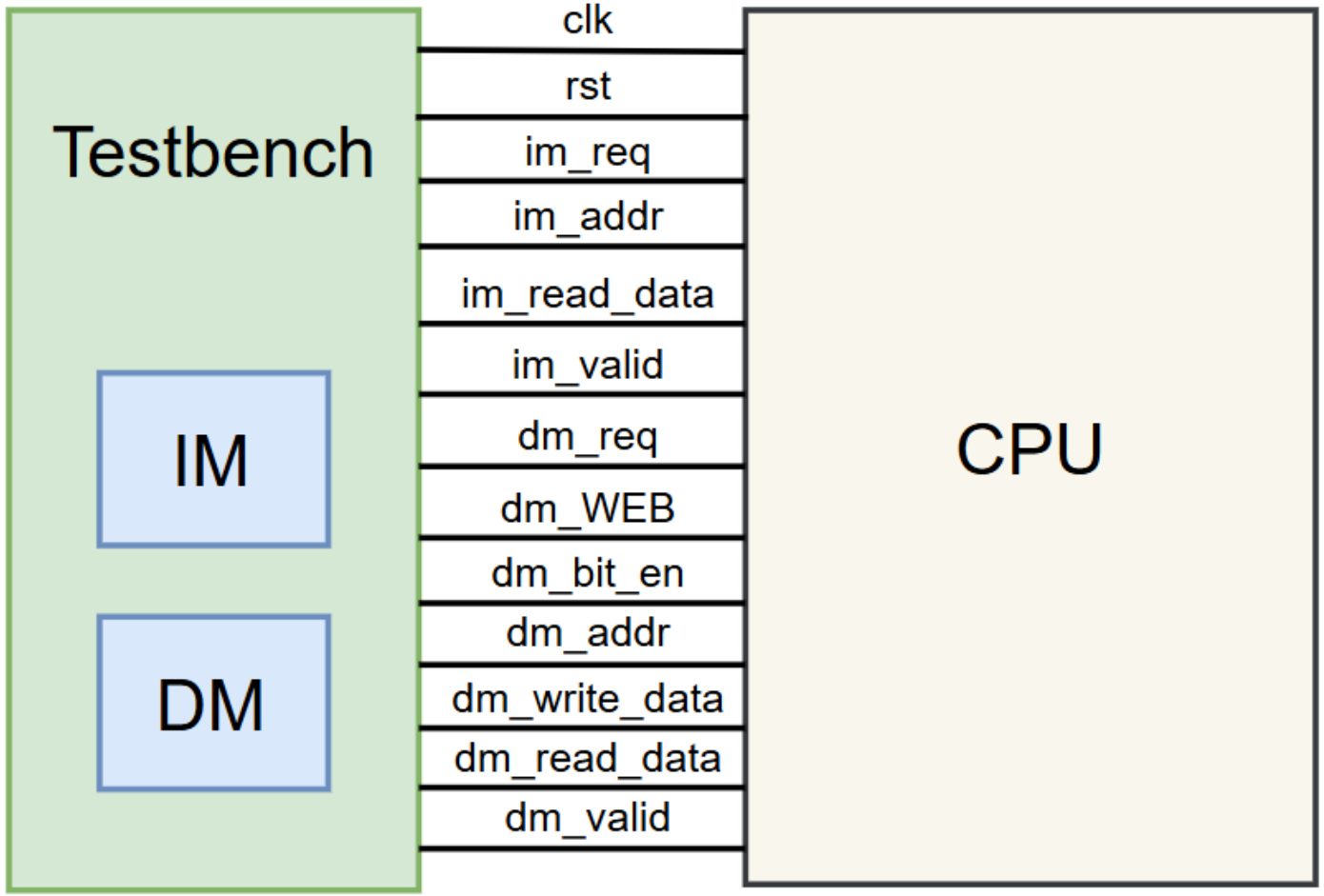
No part of this confidential report may be reproduced in any form without written permission from Prof. Lih-Yih Chiou NCKU LPHP Lab, Taiwan

Instructions DSP

31 25	24 20	19 15	14 12	11 7	6 0		
funct7	**rs2**	**rs1**	**funct3**	**rd**	**opcode**	**Mnemonic**	**Description**
0000000	rs2	rs1	000	rd	0001011	DOTP4	rd = Σ signed(rs1[8i+7:8i]) × signed(rs2[8i+7:8i]), i = 0 ~ 3
0000000	rs2	rs1	001	rd	0001011	SADD8	rd[8i+7:8i] = SAT8(signed(rs1[8i+7:8i]) + signed(rs2[8i+7:8i])), i = 0 ~ 3
0000000	rs2	rs1	010	rd	0001011	SSUB8	rd[8i+7:8i] = SAT8(signed(rs1[8i+7:8i]) - signed(rs2[8i+7:8i])), i = 0 ~ 3
0000000	rs2	rs1	011	rd	0001011	ABS	rd = (rs1 == 32'h80000000) ? 32'h7FFFFFFF : ((signed(rs1) < 0) ? -signed(rs1) : rs1)
0000000	rs2	rs1	100	rd	0001011	CLIP8	rd = (signed(rs1) > 127) ? 127 : (signed(rs1) < -128) ? -128 : signed(rs1)

No part of this confidential report may be reproduced in any form without written permission from Prof. Lih-Yih Chiou NCKU LPHP Lab, Taiwan

CPU-Testbench Interface Signals



No part of this confidential report may be reproduced in any form without written permission from Prof. Lih-Yih Chiou NCKU LPHP Lab, Taiwan

I/O Port Description (1)

Signal name	I/O	Width	Description
clk	In	1 bit	System clock
rst	In	1 bit	System reset (active high)
im_req	Out	1 bit	Instruction request from CPU (1-cycle pulse)
im_addr	Out	32 bits	Instruction address from CPU
im_read_data	In	32 bits	Instruction data to CPU
im_valid	In	1 bit	Instruction data is valid

No part of this confidential report may be reproduced in any form without written permission from Prof. Lih-Yih Chiou NCKU LPHP Lab, Taiwan

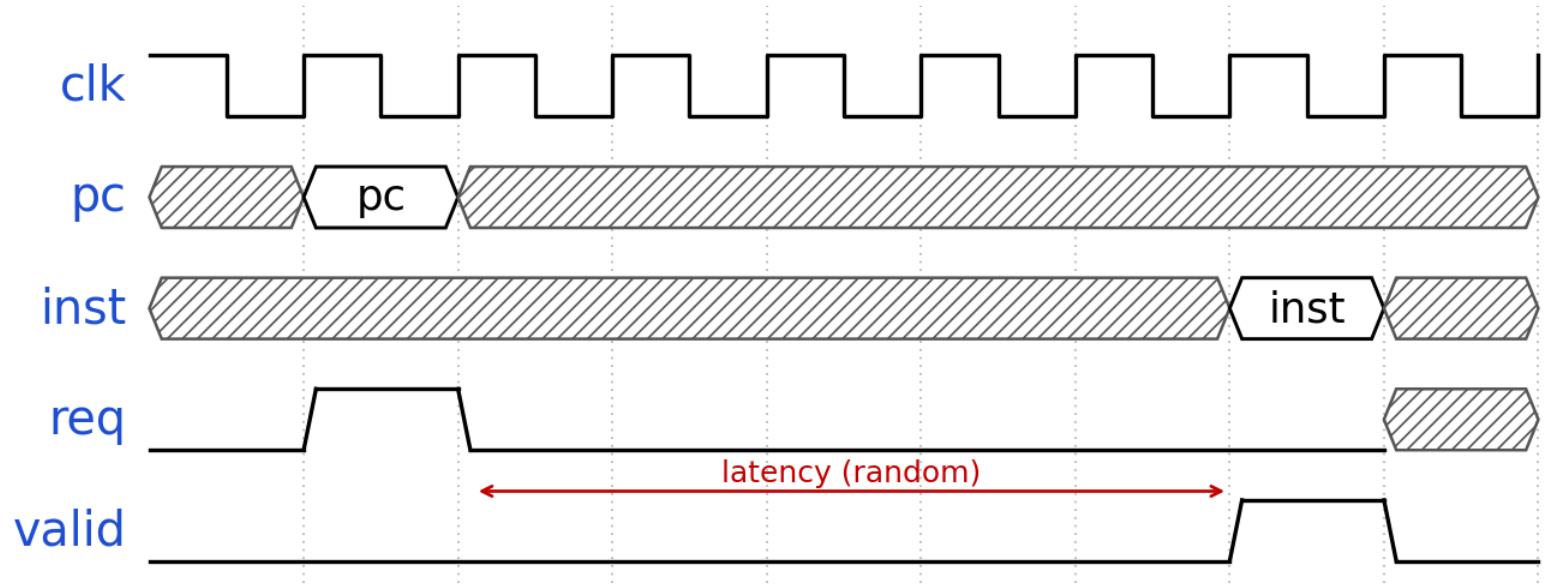
I/O Port Description (2)

Signal name	I/O	Width	Description
dm_req	Out	1 bit	Data memory request from CPU (1-cycle pulse)
dm_WEB	Out	1 bit	Write enable (0:Write, 1:Read)
dm_bit_en	Out	32 bits	Bit write mask (active low). Each 8 bits correspond to one byte.
dm_addr	Out	32 bits	Data memory address from CPU
dm_write_data	Out	32 bits	Write data from CPU
dm_read_data	In	32 bits	Read data to CPU
dm_valid	In	1 bit	Data access is valid/complete

No part of this confidential report may be reproduced in any form without written permission from Prof. Lih-Yih Chiou NCKU LPHP Lab, Taiwan

IM Read Request/Valid

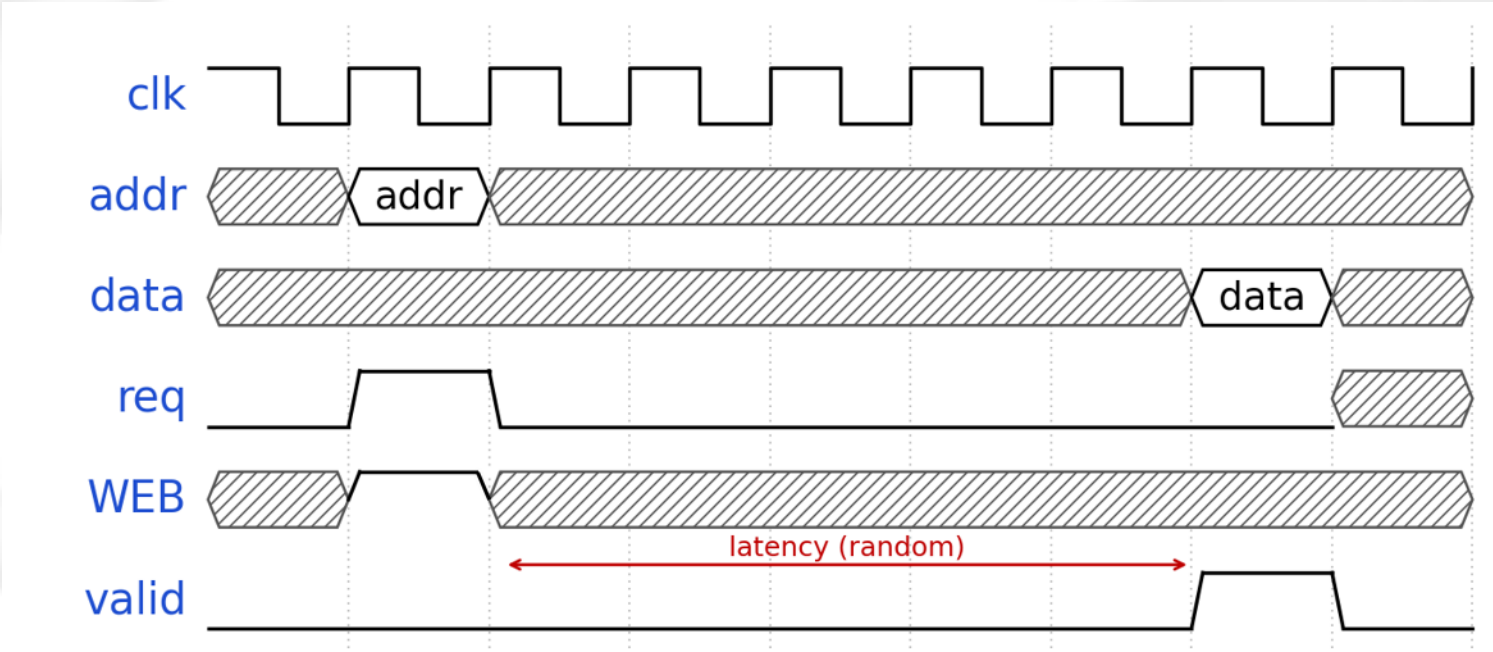
- **req** is a **1-cycle pulse**; pc is sampled only in the req cycle.
- Do not assert req again until **valid** is received.
- **valid** lasts 1 cycle and inst is valid only in that cycle. Latch it if the pipeline is stalled.



No part of this confidential report may be reproduced in any form without written permission from Prof. Lih-Yih Chiou NCKU LPHP Lab, Taiwan

DM Read Request/Valid

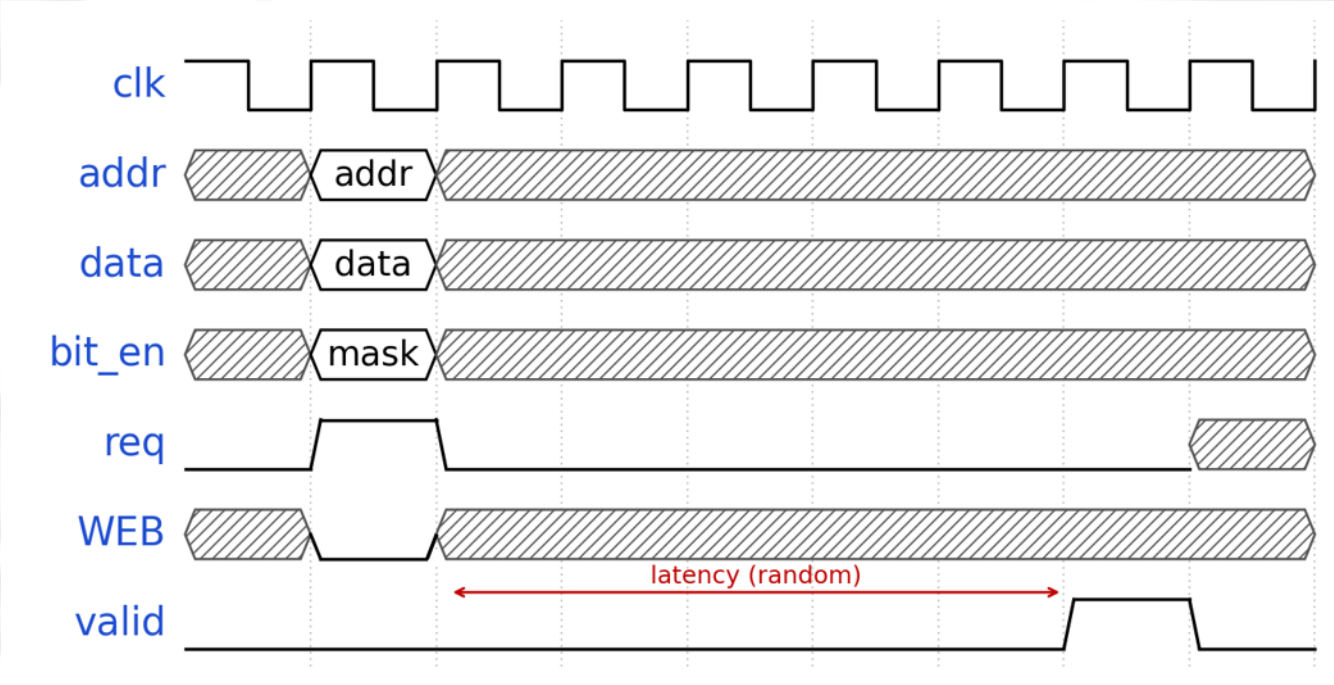
- **req** is a **1-cycle pulse**; addr and WEB are sampled only in the req cycle.
- Do not assert req again until **valid** is received.
- **valid** lasts 1 cycle and data is valid only in that cycle. Latch it if the pipeline is stalled.



No part of this confidential report may be reproduced in any form without written permission from Prof. Lih-Yih Chiou NCKU LPHP Lab, Taiwan

DM Write Request/Valid

- **req** is a **1-cycle pulse**; addr, data, bit_en and WEB are sampled only in the req cycle.
- Do not assert req again until **valid** is received.
- The data is written when **valid** is asserted (1 cycle).



No part of this confidential report may be reproduced in any form without written permission from Prof. Lih-Yih Chiou NCKU LPHP Lab, Taiwan



Test Programs

No part of this confidential report may be reproduced in any form without written permission from Prof. Lih-Yih Chiou NCKU LPHP Lab, Taiwan

Test Programs

- **Prog0**: Basic test for all 68 instructions.
- **Prog1**: Floating-point merge sort.
- **Prog2**: Test rdinstret, rdinstreth, rdcycle, and rdcycleh using a factorial C program.
- **Prog3:** Perform multiplication using the MUL instructions
- **Prog4:** Image Pixel Alpha Blending
- **Prog5:** Mini 1D-CNN
- **Prog6**: Hidden test

No part of this confidential report may be reproduced in any form without written permission from Prof. Lih-Yih Chiou NCKU LPHP Lab, Taiwan

Simulation

Situation	Command	Example
RTL simulation for progX	make rtlX	make rtl0
Post-synthesis simulation for progX	make synX	make syn1
Dump waveform (no array)	make {rtlX,synX} FSDB=1	make rtl2 FSDB=1
Dump waveform (with array)	make {rtlX,synX} FSDB=2	make syn3 FSDB=2
RTL simulation for all programs	make rtl_all	
Post-synthesis simulation for all programs	make syn_all	
Open nWave without file pollution	make nWave	
Open Superlint without file pollution	make superlint	
Count your code line in ./src	make count_lines	
Open DesignVision without file pollution	make dv	
Synthesize your RTL code	make synthesize	
Delete built files for simulation, synthesis or verification	make clean	
Check correctness of your file structure	make check	
Clean, check and pack homework to *tar* format	make tar	

No part of this confidential report may be reproduced in any form without written permission from Prof. Lih-Yih Chiou NCKU LPHP Lab, Taiwan

Simulation

- You can only modify the values in the red boxes in sim/top_tb.sv and script/DC.sdc file during post-synthesis simulation. The maximum allowed value for the clk_period is **2.0**.

DC.sdc | top_tb.sv | superlint.tcl | synthesis.tcl

sim > top_tb.sv > ...

```
1  `timescale 1ns/10ps
2  `define CYCLE 2.0
3  `define MAX 500000
4  `ifdef SYN
5  `include "top_syn.v"
```

```
set clk_period 2.0
set input_max   [expr {double(round(1000*$clk_period * 0.6))/1000}]
set input_min   [expr {double(round(1000*$clk_period * 0.0))/1000}]
set output_max  [expr {double(round(1000*$clk_period * 0.1))/1000}]
set output_min  [expr {double(round(1000*$clk_period * 0.0))/1000}]
```

- Ensure the clock period in the script/DC.sdc file matches the value defined in sim/top_tb.sv

No part of this confidential report may be reproduced in any form without written permission from Prof. Lih-Yih Chiou NCKU LPHP Lab, Taiwan



LPHPLAB
VLSI Design LAB

Grading Policy

No part of this confidential report may be reproduced in any form without written permission from Prof. Lih-Yih Chiou NCKU LPHP Lab, Taiwan

Grading policy (1)

- Simulation pass (65%) (Compile errors = 0 credit)
- Report (15%)

Program	RTL	SYN
Prog0	3%	6%
Prog1	3%	6%
Prog2	3%	6%
Prog3	3%	6%
Prog4	3%	6%
Prog5	3%	6%
Hidden Prog	4%	7%

No part of this confidential report may be reproduced in any form without written permission from Prof. Lih-Yih Chiou NCKU LPHP Lab, Taiwan

Grading policy (2)

- Performance & Area (20%)
  - PA score = (prog0 + prog1 + prog2 + prog3 + prog4 + prog5) Simulation Time * Total Cell Area (Synthesized)
  - Note: Must pass all SYN programs (incl. Hidden Prog) to join PA ranking; otherwise 0.

Rank	Credit	Rank	Credit
Top 10%	20	50 ~ 60%	10
10 ~ 20%	18	60 ~ 70%	8
20 ~ 30%	16	70 ~ 80%	6
30 ~ 40%	14	80 ~ 90%	4
40 ~ 50%	12	90 ~ 100%	2

No part of this confidential report may be reproduced in any form without written permission from Prof. Lih-Yih Chiou NCKU LPHP Lab, Taiwan

Grading policy (3)

- Deadline: 2026/10/26 15:00:00
- Late submission within 24hr → score × 0.8
- Late submission within 48hr → score × 0.6
- Late submission within 72hr → score × 0.4
- Late submission more than 72hr → assignment will NOT be accepted. (score × 0)
- Submission that fails to follow the file hierarchy and format will result in a 10-credit reduction.

No part of this confidential report may be reproduced in any form without written permission from Prof. Lih-Yih Chiou NCKU LPHP Lab, Taiwan



LPHPLAB
VLSI Design LAB

Submission Guidelines

No part of this confidential report may be reproduced in any form without written permission from Prof.
Lih-Yih Chiou NCKU LPHP Lab, Taiwan



Report

- **Summary** – Fill in Table 1 of the report template.
- **Briefly** describe how you designed your CPU and how it meets the project requirements.
- Describe **in detail** any special or unique design features of your CPU.
- **Hazard handling** – Fill in Table 2 of the report template to describe how your CPU handles the IM/DM handshake, forwarding, load-use stall, and branch flush.
- **Lessons learned** – Summarize what you learned from this homework.
- **Suggestions** – Give suggestions for improving this homework.
- Please convert your report to PDF format and submit it on Moodle. (E2XXXXXXX_HW2.pdf)

No part of this confidential report may be reproduced in any form without written permission from Prof. Lih-Yih Chiou NCKU LPHP Lab, Taiwan

File Hierarchy

```
E2XXXXXXX_HW2.tar          (Don‘t add version text, e.g. E2XXXXXXX_HW2_v1.tar)
└── E2XXXXXXX_HW2/         (Main folder of this homework)
    ├── StudentID          (Specify student ID who submit homework)
    ├── Makefile           (You shouldn’t modify it)
    ├── src/               (Your RTL code with .v or .sv format)
    │   ├── top.v or top.sv
    │   └── Other submodules (*.v or *.sv)
    ├── syn/                (Your synthesized code and timing file)
    │   ├── top_syn.v
    │   ├── top_syn.sdf
    │   ├── area_rpt.txt
    │   ├── timing_max_rpt.txt
    │   └── top_syn.ddc
    ├── script/
    │   └── Script files     (*.sdc, *.tcl, or *.setup)
    └── sim/                (Testbenches and memory libraries)
        ├── prog_lib
        ├── prog0
        ...
        ├── prog5
        ├── sim_pkg.sv
        └── top_tb.sv       (Testbench)
```

No part of this confidential report may be reproduced in any form without written permission from Prof. Lih-Yih Chiou NCKU LPHP Lab, Taiwan

Copy the Homework Environment

- Copy the homework source files
  - ➔ cp -R /home/soc_course/vsd26/vsd2600/VSD2026_HW2_v2 ./

```
superdome5:/home/soc_course/vsd26/vsd2600/ex % cp -R /home/soc_course/vsd26/vsd2600/VSD2026_HW2_v2 ./
superdome5:/home/soc_course/vsd26/vsd2600/ex % ls
VSD2026_HW2_v2/
```

No part of this confidential report may be reproduced in any form without written permission from Prof. Lih-Yih Chiou NCKU LPHP Lab, Taiwan

Homework Submission

- Open your own directory in /home/soc_course/vsd26/submission
  - ➔ mkdir /home/soc_course/vsd26/submission/$USER
- Create homework directory in your own directory
  - ➔ mkdir /home/soc_course/vsd26/submission/$USER/HW2
- Copy your homework to the designated submission directory
  - ➔ cp ./studentID_HW2.tar /home/soc_course/vsd26/submission/$USER/HW2
- You can cd to the submission directory to check your submitted files.
- Don’t modify your submit file after the deadline.

No part of this confidential report may be reproduced in any form without written permission from Prof. Lih-Yih Chiou NCKU LPHP Lab, Taiwan



LPHPLAB
VLSI Design LAB

Thanks for your participation and attendance !

No part of this confidential report may be reproduced in any form without written permission from Prof. Lih-Yih Chiou NCKU LPHP Lab, Taiwan